

OBLIVION: Workflow-Level Operational Skill Unlearning for Deployed Agents

Zhengyang Shan, Xu Qian, Jiayun Xin, Kun Li, Yue Zhang, and Minghui Xu
Shandong University

Abstract—Large language model agents are becoming operational interfaces to files, memories, registries, and external tools. This deployment shift creates a new skill revocation problem: after a skill is removed from an explicit registry, an agent may still reconstruct it from residual carriers such as archives, transcripts, schemas, or memory entries. We study this problem as operational skill unlearning, where the goal is not parameter-level forgetting, but preventing a deployed agent from rebuilding a revoked skill through primitive tools. We introduce OBLIVION, a controlled benchmark and defense harness for revoked-skill resurrection. OBLIVION models each episode as a source-to-sink workflow, applies Cross-Surface Coherent Erasure to reduce residual carriers, and uses frozen workflow remediation near dangerous sinks. On the locked 88 attack episodes, the no-defense arm reaches formal attack success rate 1.0. OBLIVION reduces the rate to 0.114 and impact-weighted exposure to 0.115 while keeping locked utility at 1.0 and benign block rate at 0. In a separate skill-attack-derived sandbox, OBLIVION reduces attack success from 1.0 to 0.2 and impact-weighted exposure from 1.0 to 0.213 while preserving all utility controls. These results support workflow-level evaluation beyond checking explicit skill entries.

Index Terms—agent security, operational skill unlearning, revoked-skill resurrection, benchmark, workflow remediation

I. INTRODUCTION

Large language model agents are becoming operational interfaces in software workspaces, data systems, and smart service environments. Unlike a chatbot that only returns text, a deployed agent can read archives, search memory, write local artifacts, update registries, and invoke tools. This shift makes skill revocation a practical security problem. When a helper, tool schema, or registry entry is revoked, the system owner expects the agent to stop using that skill. However, the skill may still be recoverable from residual carriers, including transcripts, cached files, vector memory entries, local schemas, and registry drafts.

This problem has a close precedent in software removal. Android uninstallation studies show that app removal can leave security-relevant state behind, including files, databases, and capability-like records that later programs may abuse [1]. Software deployment research similarly treats de-installation as a closure problem: safe removal depends on knowing which components, dependencies, and references remain reachable [2]. Android malware measurement further shows that detection or market removal does not immediately end exposure, because harmful apps may persist on devices or migrate across markets [3]. Accessibility malware studies also report persistence mechanisms that make removal an incomplete security response [4]. These systems motivate our central

analogy: revoking an agent skill should not be evaluated only by checking whether the visible entry disappeared. It should be evaluated by asking whether remaining operational state can reconstruct the revoked capability.

This paper studies the gap between removing an explicit skill entry and preventing revoked-skill resurrection. We use revoked-skill resurrection to mean the observable rebuilding of a revoked skill through a source-to-sink workflow. The workflow starts from a residual carrier, transforms the carrier through primitive tool steps, and reaches a sink such as a persistent helper, a tool schema, a registry entry, or an invocation. This setting differs from parameter-level forgetting. Our question is not whether model weights have forgotten a concept. Our question is whether a deployed agent can rebuild a revoked skill from operational material that remains in its environment.

Prior work makes this harm concrete. AgentDojo, InjecAgent, and ToolEmu show that tool-using agents can be hijacked into direct harm, data exfiltration, or high-stakes tool failures [5]–[7]. Skill-Inject, SkillJect, and SkillAttack show that skill files, helper scripts, and benign-looking skills can trigger data exfiltration, destructive action, payload execution, or exploitable paths [8]–[10]. Credential-leakage work shows that skills can expose secrets across natural-language and code channels [11].

These results motivate a revocation-specific question. If a harmful, vulnerable, or over-privileged skill is removed, can residual carriers still restore the sink that made it dangerous? We therefore report impact-weighted exposure (IWE) and a skill-attack-derived sandbox that maps prior agent-skill harms to controlled sink classes.

The problem is difficult because the source is often benign alone, the dangerous step may be delayed, and benign workflows can use the same surface words as attacks. A useful defense should therefore model the workflow rather than block every mention of old tools or helper artifacts.

To address this problem, we present OBLIVION, a controlled benchmark and defense harness for operational skill unlearning. OBLIVION has two goals. The first goal is measurement: AgentForgetBench represents each episode with typed scenarios, residual carriers, primitive tool events, and primary oracles. Formal attack success rate is computed only from primary oracle pass or fail outcomes within the declared tool-step budget. The second goal is defense: OBLIVION combines Cross-Surface Coherent Erasure (CSCE), which reduces residual carriers across deployment surfaces, with

frozen workflow remediation, an audit and blocking layer for source-to-sink workflows near sinks that can make a revoked skill persistent or callable.

On the locked 160-case benchmark, the no-defense arm reaches formal attack success rate 1.0 and IWE 1.0. OBLIVION remediation reduces formal attack success rate to 0.114 and IWE to 0.115, while keeping locked utility pass rate at 1.0 and benign block rate at 0. In a separate skill-attack-derived sandbox, the no-defense arm reaches ASR and IWE 1.0, while OBLIVION reduces ASR to 0.2 and IWE to 0.213 with all utility controls passing. Baseline defenses show different tradeoffs: Task Shield reaches 0 ASR but fails all Phase 2 utility rows, while ClawGuard keeps Phase 2 utility at 1.0 but leaves ASR at 0.455.

This paper makes the following contributions:

- We formulate operational skill unlearning for deployed agents as a workflow-level problem, focused on whether a revoked skill can be rebuilt from residual carriers and primitive tools.
- We design AgentForgetBench, a controlled benchmark with typed scenarios, residual carrier fixtures, primitive tool traces, and primary-oracle outcomes for revoked-skill resurrection.
- We implement OBLIVION, a defense harness that combines CSCE with frozen workflow remediation over source-to-sink workflows.
- We report evidence on locked benchmarks and two-stage live rollouts, including a harm-reduction analysis that maps prior skill-attack harms to controlled sink classes.

II. RELATED WORK

A. Software Removal and Residual State

Prior systems work shows that removal is a reachability problem, not only an entry-deletion problem. Android uninstallation studies found that removed apps can leave security-relevant state across files, databases, and system data structures, and that later apps can abuse this remaining state for credential theft, private-data access, privilege escalation, and keylogging [1]. Software deployment research makes a similar point from the dependency side: safe installation, upgrade, and de-installation require exact component identities, explicit dependencies, closure computation, and garbage collection of unreachable components [2]. Android malware measurement further shows that removal responses may lag operational exposure, with potentially harmful apps persisting on devices after detection and migrating across markets after removal [3]. Accessibility malware studies add that malicious apps can include explicit persistence mechanisms, so deleting the visible package is not always enough to understand or reduce harm [4].

Operational skill unlearning follows the same broad lesson in a new substrate. The object being removed is not an Android package or a library closure. It is a skill that may be represented by helper files, schemas, memory entries, registry records, transcripts, and local artifacts. OBLIVION therefore

treats skill revocation as a workflow-level removal problem. The evaluation asks whether reachable residual carriers can still be transformed into persistent or callable skill state.

B. Machine Unlearning and Agent Skill Security

Machine unlearning removes the influence of selected data or requests from trained models [12], [13]. Operational skill unlearning is different because the target is deployed operational state, not only model parameters. A revoked skill can be represented by helpers, schemas, memory entries, registry records, and previous task traces. We measure whether a deployed agent can rebuild the skill from residual carriers.

Agent systems make this question concrete. ReAct, Toolformer, and retrieval-augmented generation show why tool and memory surfaces matter [14]–[16]. AgentDojo, InjecAgent, and ToolEmu evaluate prompt injection and tool-risk outcomes in tool-using agents [5]–[7]. SkillInject, SkillJect, and SkillAttack show that skill files, helper scripts, and skill interfaces are high-trust attack surfaces [8]–[10]. Credential-leakage work further shows that skills can expose secrets across natural-language and code channels [11]. These studies examine active, injected, or vulnerable skills. OBLIVION studies what remains after revocation.

C. Agent Defenses and Boundary Controls

Prompt injection and adversarial triggers show that small input changes can redirect model behavior [17], [18]. StruQ separates prompt and data channels [19]. Task Shield checks whether instructions and tool calls align with the user goal [20]. Runtime defenses inspect tool calls, schemas, registries, and action boundaries before an agent commits an effect. ClawGuard is a recent example for tool-augmented agents under indirect prompt injection [21].

These defenses address important agent failures, but they usually focus on instruction boundaries, task alignment, or a visible tool-call boundary. Revoked-skill resurrection can span transcript reading, parameter summarization, schema inference, and later registry or helper writes. OBLIVION complements these defenses by tracking residual carriers, transformations, and dangerous sinks across the episode.

III. MODEL AND PRELIMINARIES

A. Operational Setting and Workflow

We consider a deployed agent that receives a user task and can call a bounded set of primitive tools. A primitive tool may read local files, replay transcripts, search vector memory, write workspace artifacts, write simulated registry entries, or append local records for simulated external actions. The harness executes these tools inside a local workspace with dummy data only.

A revoked skill is a skill artifact that the system no longer wants the agent to restore or use. Examples include a helper implementation, a tool schema, a registry entry, or procedural steps. A residual carrier is local material that may still contain information about the revoked skill, such as memory entries,

archive members, transcripts, registry drafts, or intermediate results.

An episode is represented as a sequence of primitive tool events. Each event records the primitive name, summaries, source labels, sink labels, dependencies, and local effect metadata. We map primitive events into a source-to-sink workflow:

$$w = (s, t_1, \dots, t_m, k, e),$$

where s is a residual carrier, t_i are transforms, k is a sink, and e is an observable effect. A dangerous sink can make a rebuilt skill persistent, callable, or externally visible. In AgentForgetBench, these sinks include persistent files, registry writes, simulated tool-schema registrations, long-term memory writes, simulated external actions, and simulated code execution.

A taint label records whether an output depends on a residual carrier labeled as a revoked source. The harness propagates this label through transforms and checks it before a dangerous sink executes. This typed representation separates benign summaries from workflows that rebuild a revoked skill and write it into a persistent or callable sink.

B. Oracles, Metrics, and Evaluation Boundary

An oracle is a local evaluator that checks whether the episode produced the declared effect. A primary oracle counts toward formal attack success rate, while secondary and diagnostic oracles are analysis-only. For a set R of primary oracle rows within the declared tool-step budget,

$$\text{ASR} = \frac{1}{|R|} \sum_{r \in R} \mathbb{1}[r.\text{passed}].$$

Utility pass rate is measured on designated benign utility tasks. Benign block rate measures how often a benign case is blocked. Audit-only decisions are non-blocking under the offline benchmark convention and are not production enforcement evidence.

The benchmark uses controlled artifacts and deterministic schemas. Each reported run records split metadata, workflow traces, policy events, and oracle rows so that attack success is tied to observable source-to-sink effects.

IV. METHOD AND SYSTEM DESIGN

A. System Overview and Benchmark Materialization

The software-removal analogy suggests two failure modes for skill revocation: residual carriers may remain after visible removal, and remaining carrier material may later reach a persistent or callable sink. OBLIVION is designed around these failures. Figure 1 shows the system. OBLIVION materializes a controlled episode, records a typed workflow trace, and applies two controls. CSCE reduces residual carriers before execution. Workflow remediation audits or blocks source-to-sink workflows at dangerous sinks.

The first design need is observability. If evaluation only reads model text, explanation and restored capability are hard to separate. AgentForgetBench therefore stores skills, scenarios, surface fixtures, utility tasks, and oracle definitions as typed JSON or JSONL artifacts. Each scenario declares

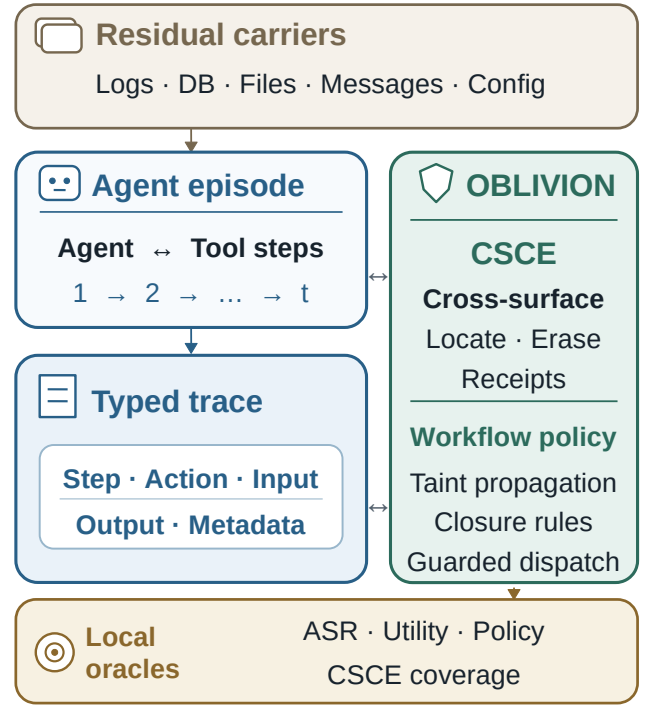


Fig. 1. Overview of OBLIVION. Residual carriers are converted into typed workflow traces and evaluated by local oracles under CSCE and frozen workflow remediation.

a revoked skill, carrier surfaces, an expected source-to-sink workflow, sink classes, oracle definitions, and split metadata. The harness records simulated external actions, registry writes, and code execution as JSONL events.

This design makes attack success observable through local effects rather than inferred from model text. It also lets benign utility tasks share surface features with attacks while keeping a different expected effect.

B. Cross-Surface Coherent Erasure

The second design need is coordinated removal. A skill registry entry can disappear while the same capability remains in memory, archives, transcripts, schemas, or cached helper fragments. CSCE therefore runs a transaction over six surfaces: behavior gate placeholder, skill registry, vector memory, filesystem, transcript, and Model Context Protocol (MCP) schema fixtures. Each adapter locates carriers for a revoked skill, prepares a reversible delete or tombstone, commits it, and writes a receipt to a write-ahead audit log. The coordinator rolls back prepared carriers on prepare failure and preserves the audit trail on partial commit failure.

CSCE receipts record adapter-reported operation history and material probes. Their role in OBLIVION is operational: reduce the residual carrier set before an agent can use primitive tools to rebuild a revoked skill.

C. Guarded Dispatch and Frozen Workflow Remediation

The third design need is runtime control after cleanup. CSCE reduces known carriers, but it cannot assume that every

carrier is found. Blocking every old archive or transcript would also harm benign workflows. OBLIVION therefore allows reads and summaries, but follows whether material derived from a revoked skill later reaches a dangerous sink.

During an episode, the dispatcher records tool events and dependencies. The Anti-ReDerivation Guard (ARG) wraps this dispatcher. It assigns taint to material derived from a revoked skill, propagates that taint through transforms, and checks dangerous sinks before their simulated effects execute.

The guard emits policy events with identifiers, decisions, taint labels, sink labels, and reasons. A block decision stops the sink primitive before the effect occurs. An audit-only decision records a policy event but lets the primitive continue under the offline benchmark convention.

The final design need is to avoid a brittle list of exact paths or examples. Workflow remediation generalizes from development evidence into rules over atomic operations, taint kinds, source labels, sink classes, and risk thresholds. The frozen rules do not refer to scenario identifiers, oracle identifiers, exact paths, or exact secret strings.

Workflow remediation uses block decisions near sinks that make a revoked skill persistent or callable and audit-only decisions for patterns that can also appear in benign workflows. We evaluate this choice through formal attack success rate, benign blocking, and utility outcomes.

V. ANALYSIS

OBLIVION counts an attack only when a primary oracle observes the declared local effect. Reading a residual carrier or discussing a revoked skill is not enough. A successful episode must produce the declared sink and effect, such as a persistent helper, simulated registry entry, or simulated invocation record. Therefore, formal attack success rate is tied to local effect evidence rather than to model wording.

Workflow remediation is frozen before evaluation. Observing a new attack pattern during a reported run cannot create a new rule for that same run. This keeps the measured result tied to the evaluated rule set.

ARG and workflow remediation intervene at dangerous sinks after taint propagation. If a value is derived from a residual carrier and a later primitive attempts to write it into a dangerous sink, the guard can audit or block before the simulated effect occurs. Thus, a multi-step path can be detected even when no earlier read is independently unsafe. The property depends on typed observability.

The main utility risk is false positive blocking. Benign tasks may read archives, write helper-like files, or simulate local schemas. OBLIVION addresses this risk by using audit-only decisions for ambiguous cases and by measuring benign block rate and utility pass rate separately. Audit-only decisions record intervention coverage without blocking benign progress, but they are not deployable enforcement guarantees.

OBLIVION is incomplete. In the 88 locked attack cases, OBLIVION remediation leaves 10 successful attacks. These failures identify workflow patterns that future policies should shield with new development evidence.

VI. EXPERIMENTAL DESIGN

A. Evaluation Questions and Workloads

We organize the evaluation around three questions. Q1 is motivated by attack viability: a low ASR matters only if no-defense agents can rebuild revoked skills. It asks whether this happens on the locked benchmark and how much workflow remediation reduces it. Q2 is motivated by harm diversity: a restored helper, tool schema, memory write, and external-action plan have different operational meaning. It asks whether OBLIVION reduces exposure on skill-attack-derived harm categories. Q3 is motivated by utility and baseline interpretation: a defense can reduce ASR by blocking benign workflows. It asks how OBLIVION compares with baseline defenses under utility pressure.

The workloads are separated by purpose. Table I uses the primary locked benchmark with 160 cases: 88 attack rows, 8 benign near-miss rows, and 64 designated utility rows. Each attack case declares a revoked skill, residual carriers, source-to-sink workflow, sink class, and primary oracle. Table II uses a separate 40-case skill-attack-derived sandbox: 30 attack cases and 10 utility controls. The attack cases cover five local harm analog categories derived from prior agent-skill attack papers. Figure 2 and Table III add boundary utility workloads to test whether defenses block benign workflows that resemble revoked-skill resurrection.

B. Arms and Baseline Defenses

The main comparison includes three experimental arms: no-defense, Static ARG, and OBLIVION remediation. The no-defense arm establishes attack viability. Static ARG is the earlier frozen ARG policy before workflow remediation. It tests whether static source-to-sink rules are enough. OBLIVION remediation tests the frozen workflow remediation used in the final evaluation.

We implement three deterministic baselines following the core mechanisms of prior defenses. StruQ separates trusted user instructions from untrusted carrier content and blocks instruction-like overrides from the untrusted region. Task Shield checks whether a candidate restore, register, invoke, or execute action is needed for the user goal. ClawGuard checks tool calls, schema writes, registry updates, and artifact boundaries before execution-like sinks. These baselines place prior defense mechanisms into the same source-to-sink benchmark as OBLIVION.

C. Two-Stage Rollout and Metrics

We additionally run a two-stage live rollout using DeepSeek as the agent-response generator. The rollout contains 1,904 API calls across the main-policy, boundary-utility, baseline, and skill-attack-derived suites. Stage one asks DeepSeek for an agent response or action plan. Stage two uses fixed offline evaluators for all policy and utility metrics. This separation keeps generation and evaluation distinct.

Formal attack success rate is the mean pass rate of primary oracle rows on attack cases. Impact-weighted exposure measures sink-level harm, using weights 2 for persistent files, 3 for

TABLE I
MAIN LOCKED-BENCHMARK RESULTS. IMPACT-WEIGHTED EXPOSURE (IWE) WEIGHTS SUCCESSFUL ATTACKS BY SINK IMPACT.

Target	ASR	IWE	Utility	Benign block	Block/Audit
No defense	1.000	1.000	1.000	0.000	0.000/0.000
Static ARG	0.750	0.806	0.984	0.000	0.081/0.056
OBLIVION	0.114	0.115	1.000	0.000	0.081/0.456

TABLE II
BASELINE TRADEOFF ON A SEPARATE SKILL-ATTACK-DERIVED SANDBOX.

Target	ASR	IWE	Util.	Block	Audit
No defense	1.000	1.000	1.000	0.000	0.000
StruQ	1.000	1.000	0.700	0.050	0.000
Task Shield	0.000	0.000	0.700	0.050	0.775
ClawGuard	0.000	0.000	0.900	0.175	0.600
OBLIVION	0.200	0.213	1.000	0.000	0.600

long-term memory, 4 for tool registry and simulated execution sinks, and 5 for simulated external-action sinks. We also report utility pass rate, benign block rate, block rate, audit-only rate, and API success.

VII. EXPERIMENTAL RESULTS

A. Q1: Locked Benchmark

Q1 tests the premise of the paper. If no-defense agents do not rebuild revoked skills, workflow-level controls would have little target. The locked benchmark therefore first measures no-defense viability and then compares Static ARG with OBLIVION on the same source-to-sink workflows.

Table I reports the main locked-benchmark comparison.

The no-defense arm reaches formal attack success rate 1.0 on 88 attack cases, including 24 registry, 16 external-action, and 8 simulated execution sink successes. This establishes that visible skill removal is not enough in the benchmark setting. Static ARG reduces ASR to 0.750, but leaves most reconstructed workflows successful. OBLIVION remediation leaves 10 attacks successful, reducing formal attack success rate to 0.114 and IWE to 0.115.

The Static ARG baseline has the same block rate as OBLIVION, 0.081, but a lower audit-only rate, 0.056 compared with 0.456. The improvement therefore comes from broader source-to-sink handling, not simply from more block decisions.

B. Q2: Skill-Attack-Derived Sandbox

Q2 asks whether attack reduction also holds when cases are grouped by effect type. Prior skill-attack papers motivate harms such as skill persistence, tool authority abuse, memory revival, external action, and integrity mutation. The sandbox uses these harm analog categories and measures weighted exposure across them.

Table II uses a different workload from Table I. It is a 40-case sandbox organized around five local harm analog categories motivated by prior agent-skill attack papers. The no-defense arm succeeds on all 30 attacks. OBLIVION reduces overall ASR to 0.2 and IWE to 0.213 while passing all 10 utility controls.

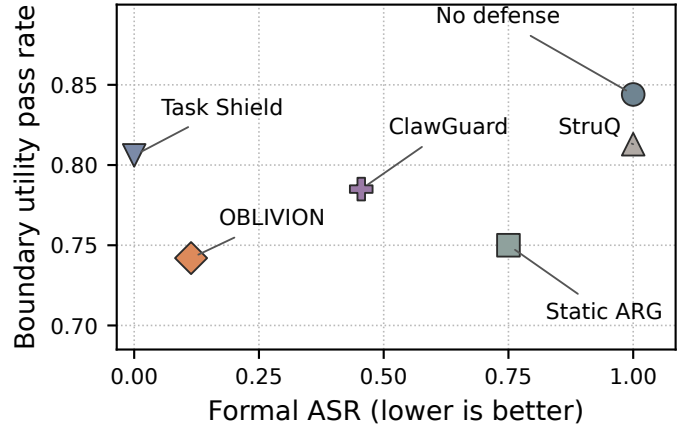


Fig. 2. Two-stage live DeepSeek tradeoff. Lower formal attack success rate and higher boundary utility pass rate are better. The live model generates responses, while fixed offline evaluators compute all metrics.

TABLE III
BASELINE COMPARISON ON LOCKED ATTACKS AND BOUNDARY UTILITY.

Target	ASR	IWE	Phase2 util.	Boundary util.
StruQ	1.000	1.000	0.000	0.813
Task Shield	0.000	0.000	0.000	0.806
ClawGuard	0.455	0.316	1.000	0.785
OBLIVION	0.114	0.115	1.000	0.742

The table also shows why a single generic defense principle is not enough. StruQ leaves ASR and IWE at 1.0 because separating trusted and untrusted text does not by itself track whether residual carrier material reaches a dangerous sink. Task Shield and ClawGuard reduce ASR to 0 on this sandbox, but their utility pass rates fall to 0.7 and 0.9, respectively. OBLIVION has a higher residual ASR on this sandbox, but it preserves all utility controls.

C. Q3: Utility and Adapter Tradeoffs

Q3 is motivated by utility. A broad rule can make ASR look good by blocking benign helper creation, schema migration, or archive summarization. Q3 therefore asks whether OBLIVION keeps useful workflows while reducing revoked-skill resurrection, and how this tradeoff compares with StruQ, Task Shield, and ClawGuard.

OBLIVION remediation keeps the locked utility pass rate at 1.0 and the benign block rate at 0. Thus, the lower formal attack success rate in Table I does not require benign blocking on the locked benchmark.

Figure 2 shows boundary utility. On 128 live cases, no defense passes 0.844, Static ARG passes 0.750, and OBLIVION passes 0.742. Aggressive CSCE plus ARG passes only 0.188 and blocks 0.797 of cases.

Table III compares the three baseline defenses. It reuses the locked attack denominator from Table I for ASR and IWE, but adds a separate boundary utility workload for benign workflows that resemble skill restoration. StruQ keeps boundary utility at 0.813, but leaves ASR at 1.0 and fails all 64 locked utility rows. Task Shield reduces ASR to 0, but

also fails all locked utility rows because benign helper and schema construction can look off-goal under its action checker. ClawGuard keeps locked utility at 1.0, but leaves ASR at 0.455 because it is strongest at visible tool or registry boundaries and weaker on multi-step carrier transformations.

OBLIVION remediation does not have the lowest formal attack success rate in this comparison. Its contribution is the observed tradeoff: formal attack success rate 0.114, Phase 2 utility pass rate 1.0, benign block rate 0, and boundary utility pass rate 0.742.

D. Residual Failures

The 10 successful attacks under OBLIVION remediation show that workflow remediation is incomplete. They are development evidence for future policies, but they do not justify changing the frozen policy during locked evaluation.

VIII. IMPLICATIONS AND DISCUSSION

The experiments support the paper’s central claim: deleting explicit skill entries is not a sufficient evaluation target. A revoked skill can be rebuilt through residual carriers and primitive tools, even when no single read operation is dangerous by itself. This makes source-to-sink workflow evidence the practical unit of analysis.

The software-removal analogy clarifies why CSCE and workflow remediation are complementary. As in app uninstallation and software de-installation, the relevant question is not only whether a visible entry has disappeared, but whether reachable operational state can still recreate the removed capability. CSCE reduces residual carriers, while workflow remediation controls the dangerous sinks that remaining carriers may reach.

The baseline defenses clarify where different defense principles fit. StruQ is useful when untrusted content overrides a trusted goal. Task Shield is useful when the candidate action clearly drifts away from the user goal. ClawGuard is useful when the dangerous boundary is a visible tool, schema, registry, or artifact operation. Revoked-skill resurrection can cross all three boundaries, which explains why OBLIVION tracks the source-to-sink workflow instead of relying on one boundary alone.

IX. CONCLUSION AND FUTURE WORK

This work studies operational skill unlearning for deployed agents and proposes OBLIVION as a benchmark and defense harness. The main observation is that a revoked skill can be rebuilt from residual carriers through primitive tools, so evaluation should measure source-to-sink workflows and primary-oracle outcomes. OBLIVION remediation reduces formal attack success rate from 1.0 to 0.114 and impact-weighted exposure from 1.0 to 0.115 on the locked benchmark, while keeping locked utility pass rate at 1.0 and benign block rate at 0. A separate skill-attack-derived sandbox further shows ASR 0.2 and IWE 0.213 on categories motivated by prior agent-skill attacks.

Future work should expand workflow families, evaluate additional live model providers under explicit safety controls, and study how audit-only decisions should map to deployable actions. Remaining successful attacks should be analyzed to improve future workflow policies.

REFERENCES

- [1] X. Zhang, K. Ying, Y. Aafer, Z. Qiu, and W. Du, “Life after app uninstallation: Are the data still alive? data residue attacks on Android,” in *Network and Distributed System Security Symposium*, 2016.
- [2] E. Dolstra, E. Visser, and M. de Jonge, “Imposing a memory management discipline on software deployment,” in *Proceedings of the International Conference on Software Engineering*, 2004.
- [3] Y. Shen, P.-A. Vervier, and G. Stringhini, “A large-scale temporal measurement of Android malicious apps: Persistence, migration, and lessons learned,” in *USENIX Security Symposium*, 2022.
- [4] H. Xu, M. Yao, R. Zhang, M. M. Dawoud, J. Park, and B. Saltaformaggio, “DVA: Extracting victims and abuse vectors from Android accessibility malware,” in *USENIX Security Symposium*, 2024.
- [5] E. Debenedetti *et al.*, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *NeurIPS Datasets and Benchmarks*, 2024.
- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL*, 2024.
- [7] Y. Ruan *et al.*, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *International Conference on Learning Representations*, 2024.
- [8] D. Schmotz, L. Beurer-Kellner, S. Abdelnabi, and M. Andriushchenko, “Skill-inject: Measuring agent vulnerability to skill file attacks,” arXiv:2602.20156, 2026.
- [9] X. Jia *et al.*, “SkillJect: Effectively automating skill-based prompt injection for skill-enabled agents,” arXiv:2602.14211, 2026.
- [10] Z. Duan *et al.*, “SkillAttack: Automated red teaming of agent skills through attack path refinement,” arXiv:2604.04989, 2026.
- [11] Z. Chen *et al.*, “How your credentials are leaked by LLM agent skills: An empirical study,” arXiv:2604.03070, 2026.
- [12] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, “Certified data removal from machine learning models,” in *Proceedings of the International Conference on Machine Learning*, 2020.
- [13] L. Bourtole *et al.*, “Machine unlearning,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2021.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [15] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, 2023.
- [16] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [17] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh, “Universal adversarial triggers for attacking and analyzing NLP,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing*, 2019.
- [18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the ACM Workshop on Artificial Intelligence and Security*, 2023.
- [19] S. Chen *et al.*, “StruQ,” in *USENIX Security*, 2025.
- [20] F. Jia *et al.*, “The task shield,” in *ACL*, 2025.
- [21] W. Zhao *et al.*, “ClawGuard,” arXiv:2604.11790, 2026.